

Redes de Computadoras — TP Teórico: World Wide Web

Problema 1

Página 1 — Núcleo básico (lista numerada):

```
<!DOCTYPE html>
<html lang="es">
<head><meta charset="UTF-8"><title>Núcleo básico obligatorio</title></head>
<body>
  
  <b>Universidad Nacional de Quilmes</b>
  <h1><i>Tecnicatura en Programación Informática</i></h1>
  <h3><i><u>Materias del núcleo básico obligatorio:</u></i></h3>
  <ol>
    <li>Introducción a la Programación</li>
    <li>Organización de Computadoras</li>
    <li>Matemática I</li>
    <li>Programación con Objetos I</li>
    <li>Bases de Datos</li>
    <li>Estructuras de Datos</li>
    <li>Programación con objetos II</li>
  </ol>
  <p style="text-align:right">
    <a href="http://www.unq.edu.ar/">Volver a la página principal de la UNQ</a>
  </p>
</body>
</html>
```

Página 2 — Núcleo avanzado (tabla):

```
<!DOCTYPE html>
<html lang="es">
<head><meta charset="UTF-8"><title>Núcleo avanzado obligatorio</title></head>
<body>
  
  <b>Universidad Nacional de Quilmes</b>
  <h1><i>Tecnicatura en Programación Informática</i></h1>
  <h3><i><u>Materias del núcleo avanzado obligatorio:</u></i></h3>
  <p>Las materias del núcleo avanzado obligatorio completan la formación obligatoria del estudiante. Para todas las materias incluidas en la siguiente tabla, el régimen de cursado es cuatrimestral, y la modalidad es presencial.</p>
  <table border="1">
    <tr><th>Materia</th><th>Horas semanales</th><th>Carga horaria</th><th>Créditos</th></tr>
    <tr><td>Redes de Computadoras</td><td>6</td><td>108</td><td>12</td></tr>
    <tr><td>Sistemas Operativos</td><td>6</td><td>108</td><td>12</td></tr>
```

```
|<td>Programación Concurrente</td><td>4</td><td>72</td><td>8</td></tr>
|<td>Matemática II</td><td>4</td><td>72</td><td>8</td></tr>
</table>
<p style="text-align:right">
  <a href="http://www.unq.edu.ar/">Volver a la página principal de la UNQ</a>
</p>
</body>
</html>

|  |

|  |

```

Página 3 — Núcleo complementario (formulario con botón de búsqueda):

```

<!DOCTYPE html>
<html lang="es">
<head><meta charset="UTF-8"><title>Núcleo complementario</title></head>
<body>
  
  <b>Universidad Nacional de Quilmes</b>
  <h1><i>Tecnicatura en Programación Informática</i></h1>
  <h3><i><u>Asignaturas del núcleo complementario:</u></i></h3>
  <p>Las materias del núcleo complementario permiten orientar al estudiante
hacia un
  perfil determinado dentro del universo amplio y cambiante de los proyectos
de
  desarrollo de software.</p>
  <form action="http://www.sistemas.unq.edu/cgi-bin/buscar-materias.exe"
method="get">
    <input type="text" name="busqueda">
    <input type="image" src="http://www.unq.edu.ar/images/ico_search.gif"
alt="Buscar">
  </form>
  <p style="text-align:right">
    <a href="http://www.unq.edu.ar/">Volver a la página principal de la UNQ</a>
  </p>
</body>
</html>

```

Problema 2

```

GET / HTTP/1.1
Host: www.dc.uba.ar
HEAD /tac HTTP/1.1
Host: www.dc.uba.ar
GET /logo.jpg HTTP/1.1
Host: www.dc.uba.ar
If-Modified-Since: Sun, 01 Jun 2025 00:00:00 GMT

```

GET trae el recurso completo; HEAD trae solo los encabezados (metadatos sin cuerpo); If-Modified-Since hace un GET condicional: si no cambió desde esa fecha el servidor responde 304 Not Modified (sin cuerpo); si cambió, responde 200 con el archivo.

Problema 3

Ambas abren TCP al mismo servidor físico (www.inta.gob.ar:80) y piden el recurso /. La diferencia está en el encabezado Host:, que el servidor usa para el hosting virtual (varios sitios en una misma IP):

- 1ª: Host: www.inta.gob.ar → devuelve la página del INTA.
- 2ª: Host: www.mercosurt.org.ar → le pide al servidor del INTA el sitio de mercosurt. Si ese servidor también aloja ese dominio, devuelve esa otra página; si no, responde un error (404/400) o el sitio por defecto.

Conclusión: con un mismo IP+puerto el servidor decide qué sitio servir según Host:. Por eso Host es obligatorio en HTTP/1.1.

Problema 4

1. 1 servidor web alcanza.
2. 1 dirección IP.
3. 1 puerto TCP (el 80). Cada cliente se distingue por la cupla de 4 (IP origen, puerto origen, IP destino, puerto destino), no por un puerto nuevo en el servidor.
4. Con 150.000.000 de clientes: en teoría sigue siendo 1 IP y 1 puerto (el modelo de sockets lo soporta). Lo que cambia es la capacidad: hacen falta varios servidores detrás de un balanceador, réplicas, varias IPs por DNS, etc. Es un problema de escalabilidad/carga, no de direccionamiento.

Problema 5

- HTTP/1.1 200 OK → pedido exitoso.
- Como fue HEAD, solo vienen encabezados, sin cuerpo.
- Content-Length: 286 → la página pesa 286 bytes. Content-Type: text/html → es HTML.
- Last-Modified → última modificación (sirve para caché / If-Modified-Since).
- Server: Microsoft-IIS/7.0 y X-Powered-By: ASP.NET → corre sobre IIS con ASP.NET.
- ETag → identificador de versión del recurso; Accept-Ranges: bytes → admite descargas parciales (rangos).

Problema 6

- HTTP/1.0 (no persistente): una conexión por objeto → 7 conexiones.
- HTTP/1.1 (persistente): una conexión por servidor → 3 conexiones (server1, server2, server3).

Problema 7

Son 3 servidores distintos (el del sitio + 2 de imágenes). HTTP/1.1 reutiliza la conexión por servidor → 3 conexiones TCP.

Problema 8

Objetos que el navegador descarga: el HTML + logo_transp.gif (unq) + mail_logo.png (google) = 3 objetos. Los <a href> son enlaces, no se descargan solos; solo cuentan los src.

→ HTTP/1.0 no persistente = 3 conexiones TCP.

Problema 9

Objetos (todos en el mismo servidor): index.html + style.css + searchline.png + home.png + search_icon.gif = 1 base + 4 embebidos.

- HTTP/1.0 (no persistente, secuencial): 2 RTT por objeto (1 handshake + 1 pedido/respuesta) → base 2 RTT + 4×2 RTT = 10 RTT.
- HTTP/1.1 persistente con pipelining: 1 RTT handshake + 1 RTT por la base + 1 RTT para los 4 embebidos en paralelo = 3 RTT. (Sin pipelining: 1 + 1 + 4 = 6 RTT.)

Problema 10

Supuestos: HTTP/1.1 persistente, pedidos secuenciales (1 RTT por objeto), handshake despreciable. HIT en proxy = RTT host-proxy = 10 ms. MISS = host-proxy + proxy-origen = 10+300 = 310 ms (Web) / 10+400 = 410 ms (Ads). Los .php y cada .html son páginas distintas → siempre MISS; las imágenes quedan cacheadas tras el primer pedido.

a) index.html (caché vacía): todo MISS → 310 + (4×310) + (3×410) = 2780 ms. Quedan en caché: index, 1-4.jpg, 1-3.gif.

b) comprar.php + 3.jpg,4.jpg,5.jpg + 2.gif,3.gif: 310 (php miss) + 10 (3.jpg hit) + 10 (4.jpg hit) + 310 (5.jpg miss) + 10 (2.gif hit) + 10 (3.gif hit) = 660 ms. Se agrega 5.jpg a caché.

c) gracias-por-comprar.html + 1.jpg,4.jpg,6.jpg + 1.gif,3.gif: 310 (html miss) + 10 (1.jpg hit) + 10 (4.jpg hit) + 310 (6.jpg miss) + 10 (1.gif hit) + 10 (3.gif hit) = 660 ms.